

Alma Compound Object Handling

Overview

This document describes the compound object (parent/child) relationship handling in the Manage Digital Ingest application for Alma mode workflows. It covers both:

1. **Automatic Creation:** CSV generation with automatic compound detection from file naming patterns
2. **Manual Processing:** UpdateCSV view processing of existing compound relationships

Implementation Dates

- Manual Processing in UpdateCSV: November 4, 2025
- Automatic Creation in CSV Generator: January 14, 2026

Purpose

In Alma Digital workflows, compound objects represent collections of related digital items (e.g., multi-page documents, photo albums, multi-part works). These require special metadata handling to:

- Link parent and child records through a common `group_id`
- Build a Table of Contents (TOC) from child titles
- Set appropriate object types and representations
- Validate minimum child count requirements

Part 1: Automatic Compound Object Creation

Overview

The CSV Generator (Storage view) automatically detects and creates compound objects based on file naming patterns. When generating CSV rows from selected files, the system analyzes filenames to identify groups of related files that should be treated as a compound object.

Automatic Detection Logic

File Naming Patterns Recognized

The system detects compound objects using two primary patterns:

Pattern 1: Explicit Numbering with Underscore or Space

Files ending with `<number>` or `<space><number>`:

```
MyDocument_1.pdf
MyDocument_2.pdf
MyDocument_3.pdf
```

or

```
MyDocument 1.pdf
MyDocument 2.pdf
MyDocument 3.pdf
```

A mix of numbers preceeded by a space or underscore should also work.

Pattern 2: Implicit Part 1 + Numbered Siblings

A base filename without numbering + numbered files with the same base:

```
MyDocument.pdf      ← Treated as Part 1
MyDocument_2.pdf    ← Part 2
MyDocument_3.pdf    ← Part 3
```

Detection Algorithm

The detection process follows these steps:

1. **First Pass - Pattern Analysis**

- Scan all selected filenames
- Use regex pattern: `^(.+)[_ ](\d+)$` to extract base name and part number
- Files matching pattern: Group by base name
- Files not matching: Store as potential implicit Part 1 files

2. **Second Pass - Implicit Part 1 Detection**

- For each stored potential Part 1 file
- Check if its base name matches any grouped files
- If match found: Insert as Part 1 of that compound group
- If no match: Treat as standalone file

3. **Validation**

- Only create compound objects for groups with **2 or more parts**
- Groups with only 1 part are treated as standalone files

4. **Sorting**

- All parts within each compound group are sorted by part number
- Ensures correct sequential ordering

Examples

Example 1: Underscore Numbering

Input Files:

```
Album_1.tif
Album_2.tif
Album_3.tif
Album_4.tif
```

Result: Creates compound object "Album" with 4 children

Example 2: Implicit Part 1

Input Files:

```
Report.pdf
Report_2.pdf
Report_3.pdf
```

Result: Creates compound object "Report" with 3 children (Report.pdf becomes Part 1)

Example 3: Mixed Files

Input Files:

```
Book_1.jpg
Book_2.jpg
Photo.png
Letter_1.pdf
```

Result:

- Compound object "Book" with 2 children
- Standalone file "Photo.png"
- Standalone file "Letter_1.pdf" (only 1 part, doesn't meet minimum)

CSV Structure Created

Parent Row Generation

For each detected compound group, a parent row is created with:

Field	Value	Description
<code>originating_system_id</code>	Generated unique ID (dg_*)	Parent's unique identifier
<code>group_id</code>	Same as parent's originating_system_id	Links family together
<code>dc:identifier</code>	Handle URL with numeric portion	Persistent identifier

Field	Value	Description
dc:title	Base name from filename	Parent title (e.g., "Album")
dc:type	"compound"	Identifies as compound object
compoundrelationship	"parent:<basename>"	Marks as parent record
dcterms:tableOfContents	Pipe-separated child titles	Auto-generated TOC

Child Row Generation

For each part in the compound group, a child row is created with:

Field	Value	Description
file_name_1	dg_*.<ext>	Renamed file with unique ID
originating_system_id	Generated unique ID (dg_*)	Child's unique identifier
group_id	Parent's originating_system_id	Links to parent
dc:identifier	Handle URL with numeric portion	Persistent identifier
dc:title	"<basename> - Part <N>"	Descriptive title
compoundrelationship	"child:part<N>"	Marks as child with part number
rep_label	Same as dc:title	Representation label
rep_public_note	Same as dc:type	Representation note

Automatic File Renaming

The system also automatically renames temporary files to match the dg_* convention:

1. Before Processing:

```
storage/temp/Album_1.tif
storage/temp/Album_2.tif
```

2. After Processing:

```
storage/temp/dg_1234567890.tif  (Album_1.tif renamed)
storage/temp/dg_1234567891.tif  (Album_2.tif renamed)
```

3. Session Updates:

- temp_file_info updated with new paths
- temp_files list updated
- selected_file_paths updated

Complete Example

Input

User selects these files in File Selector:

```
Photos_1.jpg
Photos_2.jpg
Photos_3.jpg
```

Processing

- System detects pattern Photos_<number>
- Groups all 3 files under base name "Photos"
- Validates: 3 parts ≥ 2 (passes minimum requirement)
- Generates parent + 3 children rows

Generated CSV Output

Parent Row:

```
originating_system_id: dg_1736950000
group_id: dg_1736950000
dc:identifier: http://hdl.handle.net/11084/1736950000
dc:title: Photos
dc:type: compound
compoundrelationship: parent:Photos
dcterms:tableOfContents: Photos - Part 1 | Photos - Part 2 | Photos - Part 3
```

Child Row 1:

```
file_name_1: dg_1736950001.jpg
originating_system_id: dg_1736950001
group_id: dg_1736950000
dc:identifier: http://hdl.handle.net/11084/1736950001
dc:title: Photos - Part 1
compoundrelationship: child:part1
rep_label: Photos - Part 1
```

Child Row 2:

```
file_name_1: dg_1736950002.jpg
originating_system_id: dg_1736950002
group_id: dg_1736950000
dc:identifier: http://hdl.handle.net/11084/1736950002
dc:title: Photos - Part 2
compoundrelationship: child:part2
rep_label: Photos - Part 2
```

Child Row 3:

```
file_name_1: dg_1736950003.jpg
originating_system_id: dg_1736950003
group_id: dg_1736950000
dc:identifier: http://hdl.handle.net/11084/1736950003
dc:title: Photos - Part 3
compoundrelationship: child:part3
rep_label: Photos - Part 3
```

Logging

The system logs all compound object detection:

```
[INFO] Detected compound object 'Photos' with 3 parts
[INFO] Detected implicit part 1 for compound 'Report': Report.pdf
[INFO] Renamed temp file: Photos_1.jpg -> dg_1736950001.jpg
[INFO] Renamed temp file: Photos_2.jpg -> dg_1736950002.jpg
[INFO] Renamed temp file: Photos_3.jpg -> dg_1736950003.jpg
```

Code Location

- **File:** [views/storage_view.py](#)
- **Method:** `generate_csv_rows()`
- **Lines:** Approximately 64-210

Best Practices

Recommended Naming Conventions

 Good:

```
Document_1.pdf, Document_2.pdf, Document_3.pdf
Album 1.jpg, Album 2.jpg, Album 3.jpg
Book.pdf, Book_2.pdf, Book_3.pdf (implicit Part 1)
```

 Avoid:

```
Document-1.pdf (hyphen not recognized)
Document.part1.pdf (pattern not recognized)
Doc1.pdf, Doc2.pdf (no separator before number)
```

Tips

1. **Consistent Separators:** Use either underscore or space consistently
2. **Sequential Numbers:** Start from 1 and increment sequentially
3. **Same Base Name:** All parts must share the exact same base name
4. **File Extensions:** Can be different across parts if needed
5. **Minimum Parts:** Ensure at least 2 parts for a valid compound

Part 2: Manual Compound Processing in UpdateCSV

Overview

The UpdateCSV view provides manual processing of compound object relationships that have already been defined in the CSV (either created automatically by the CSV Generator or added manually by the user). This processing enriches the metadata and validates the compound structure.

Functionality

When It Runs

- **Mode:** Alma only
- **Timing:** Step 3.65 in the `apply_all_updates()` method
- **Location:** After Handle URL processing
- **Trigger:** Automatic when "Apply All Updates" button is clicked
- **Condition:** Only runs if `compoundrelationship` column exists in CSV

Required CSV Columns

Essential:

- `compoundrelationship`: Identifies parent ("parent:...") and child ("child:...") records
- `originating_system_id`: Unique identifier for each record
- `group_id`: Links parent and children together (populated by this logic)

Optional but recommended:

- `dc:title`: Title of each object (used in TOC)
- `dc:type`: Type of each object (used in TOC)
- `dcterms:tableOfContents`: Where parent's TOC is stored
- `dcterms:type.dcterms:DCMIType`: Cleared for parent records
- `rep_label`: Set to child's title
- `rep_public_note`: Set to child's type
- `mms_id`: Marked with error if validation fails

Detection Logic

The logic identifies compound objects by scanning the CSV sequentially:

1. **Parent Detection:** Finds rows where `compoundrelationship` starts with "parent"
2. **Child Detection:** Processes immediately following rows that start with "child"
3. **Grouping:** All consecutive children belong to the parent above them

Processing Steps

For Each Parent Record:

1. Group Identification

- Set parent's `group_id` = parent's `originating_system_id`
- This creates the identifier that links the family together

2. Child Discovery

- Scan forward through CSV rows
- Collect all consecutive rows where `compoundrelationship` starts with "child"
- Stop when encountering non-child row or end of CSV

3. Child Validation

- Count total children found

- **Requirement:** Minimum 2 children per parent
- If < 2 children: Log error and mark parent's `mms_id` = "ERROR Too few children!"

4. Table of Contents Construction

- For each valid child, build TOC entry:
 - Format with both title and type: "Title (Type) | "
 - Format with title only: "Title | "
- Concatenate all entries with pipe separators
- Store in parent's `dcterms:tableOfContents` field

5. Parent Metadata Updates

- Set `dc:type` = "compound"
- Clear `dcterms:type.dcterms:DCMIType` = ""
- Store complete TOC string

For Each Child Record:

1. Group Membership

- Set child's `group_id` = parent's `originating_system_id`
- Links child to parent through common identifier

2. Representation Fields

- Set `rep_label` = child's `dc:title`
- Set `rep_public_note` = child's `dc:type`
- These fields describe the digital representation

Example Workflow

Input CSV:

Row	compoundrelationship	originating_system_id	dc:title	dc:type	group_id
1	parent:album	dg_1234567890	Summer Album	Collection	
2	child:page1	dg_1234567891	Page 1	StillImage	
3	child:page2	dg_1234567892	Page 2	StillImage	
4	child:page3	dg_1234567893	Page 3	StillImage	

After Processing:

Row	compoundrelationship	originating_system_id	dc:title	dc:type	group_id	dcterms:tableOfContents	rep_label	rep_public_
1	parent:album	dg_1234567890	Summer Album	compound	dg_1234567890	Page 1 (StillImage) Page 2 (StillImage) Page 3 (StillImage)		
2	child:page1	dg_1234567891	Page 1	StillImage	dg_1234567890		Page 1	StillImage
3	child:page2	dg_1234567892	Page 2	StillImage	dg_1234567890		Page 2	StillImage
4	child:page3	dg_1234567893	Page 3	StillImage	dg_1234567890		Page 3	StillImage

Changes Made:

-  Parent `group_id` set to its own `originating_system_id`
-  All children `group_id` set to parent's `originating_system_id`
-  Parent `dc:type` changed to "compound"
-  Parent `dcterms:tableOfContents` populated with child information
-  Each child's `rep_label` and `rep_public_note` set

Logging

Comprehensive logging tracks all processing:

```
[INFO] Processing Alma compound parent/child relationships...
[INFO] Found parent at row 1 with originating_system_id: dg_1234567890
[INFO] Set parent group_id to: dg_1234567890
[INFO] Processed child at row 2: Page 1
[INFO] Processed child at row 3: Page 2
[INFO] Processed child at row 4: Page 3
[INFO] Set parent TOC: Page 1 (StillImage) | Page 2 (StillImage) | Page 3 (StillImage)
```

```
[INFO] Set parent dc:type to 'compound'
[INFO] Processed 1 compound parent/child group(s)
```

Error Logging:

```
[ERROR] *ERROR* Parent at row 1 has only 1 child(ren), need at least 2!
```

Edge Cases Handled

1. **Missing Columns:** Safely checks for column existence before updating
2. **Insufficient Children:** Validates minimum 2 children, logs error if not met
3. **Empty Values:** Handles empty/missing titles and types gracefully
4. **Non-String Values:** Converts all values to strings for comparison
5. **Sequential Processing:** Correctly handles multiple parent/child groups in one CSV
6. **Orphaned Children:** Only processes children immediately following a parent

Validation Rules

Parent Record Requirements:

-  Must have `compoundrelationship` starting with "parent"
-  Must have valid `originating_system_id`
-  Must have at least 2 children immediately following
-  Parent with 0-1 children = validation error

Child Record Requirements:

-  Must have `compoundrelationship` starting with "child"
-  Must immediately follow parent or another child
-  Should have `dc:title` for TOC (optional but recommended)

Code Location

- **File:** `views/update_csv_view.py`
- **Method:** `apply_all_updates()`
- **Step:** 3.65 (Alma compound parent/child processing)
- **Lines:** Approximately 365-445 (may vary with updates)

Reference Implementation

This logic is based on the reference implementation in:

- **Source:** `../migrate-MODS-to-dcterms/manage-collections.py`
- **Lines:** 312-373
- **Adaptation:** Converted from Polars DataFrames to Pandas DataFrames

Key differences from reference:

- Uses Pandas `.at[]` accessor instead of Polars
- Simplified dginfo handling (not yet fully implemented)
- Enhanced logging for debugging
- Added validation and error handling

Integration with Alma Digital

After CSV processing, the resulting metadata structure works with Alma Digital's compound object model:

1. **Import Process:** Alma recognizes compound objects by matching `group_id` values
2. **Parent Display:** Shows TOC with clickable links to children
3. **Child Access:** Each child becomes a separate digital representation
4. **Navigation:** Users can browse between related items in the compound

Future Enhancements

Potential improvements for consideration:

1. **dginfo Implementation:** Full JSON-based digital information tracking
2. **Flexible Validation:** Configurable minimum child count
3. **TOC Formatting:** Customizable TOC templates
4. **Nested Compounds:** Support for grandparent/parent/child hierarchies
5. **Post-Processing Report:** Summary of all compound groups created
6. **Error Recovery:** Attempt to fix common issues automatically

Complete Workflow: From Files to Alma

End-to-End Process

The complete compound object workflow in the Manage Digital Ingest application:

Step 1: File Selection

1. User selects multiple files in File Selector view
2. Files can have naming patterns like:
 - Album_1.jpg, Album_2.jpg, Album_3.jpg
 - Report.pdf, Report_2.pdf, Report_3.pdf

Step 2: Automatic CSV Generation

1. Navigate to Storage view (CSV Generator)
2. Click "Generate CSV Rows from Selected Files"
3. **Automatic detection occurs:**
 - System analyzes all filenames
 - Identifies compound groups (≥ 2 parts with same base name)
 - Creates parent row for each compound group
 - Creates child rows for each part
 - Populates **compoundrelationship** column automatically
 - Sets initial **group_id** values
 - Builds initial Table of Contents
4. CSV displayed in data table

Step 3: Export and Load into UpdateCSV

1. Click "Export CSV" to save generated CSV
2. Navigate to UpdateCSV view
3. Load the exported CSV file

Step 4: Manual Compound Processing

1. Add/edit metadata as needed
2. Click "Apply All Updates"
3. **Automatic enrichment occurs:**
 - System validates compound structure
 - Verifies minimum 2 children per parent
 - Enriches Table of Contents with type information
 - Sets representation fields (rep_label, rep_public_note)
 - Validates **group_id** consistency
 - Logs any errors or warnings

Step 5: Final Export

1. Export the enriched CSV
2. CSV is ready for Alma Digital import

Integration Points

Between CSV Generator and UpdateCSV

The CSV Generator creates the foundation:

```
compoundrelationship,originating_system_id,group_id,dc:title,dc:type,dcterms:tableOfContents
parent:Album,dg_1234567890,dg_1234567890,Album,compound,"Album - Part 1 | Album - Part 2"
child:part1,dg_1234567891,dg_1234567890,Album - Part 1,,
child:part2,dg_1234567892,dg_1234567890,Album - Part 2,,
```

UpdateCSV enriches it:

```
compoundrelationship,originating_system_id,group_id,dc:title,dc:type,dcterms:tableOfContents,rep_label,rep_public_note
parent:Album,dg_1234567890,dg_1234567890,Album,compound,"Album - Part 1 (StillImage) | Album - Part 2 (StillImage)",,
child:part1,dg_1234567891,dg_1234567890,Album - Part 1,StillImage,,Album - Part 1,StillImage
child:part2,dg_1234567892,dg_1234567890,Album - Part 2,StillImage,,Album - Part 2,StillImage
```

Workflow Advantages

Automatic Creation Benefits:

- **✓ Speed:** Instantly creates compound structure from filenames
- **✓ Consistency:** Uniform naming and ID generation
- **✓ Accuracy:** No manual typing errors in relationships
- **✓ Convenience:** Works with existing file naming practices

Manual Processing Benefits:

- **✓ Validation:** Ensures compound structure meets requirements
- **✓ Enrichment:** Adds detailed metadata to TOC
- **✓ Flexibility:** Allows manual adjustment before final processing
- **✓ Error Detection:** Identifies and flags structural issues

Hybrid Approach

Users can also combine automatic and manual methods:

1. **Auto-generate** some compounds from file naming
2. **Manually add** additional compounds to the CSV
3. **Process all together** in UpdateCSV

All compounds are processed consistently regardless of their origin.

Troubleshooting

Automatic Creation Issues (CSV Generator)

Problem: Files not being grouped as compound

- **Check:** Verify filenames follow pattern `basename_1, basename_2` or `basename 1, basename 2`
- **Check:** Ensure exactly the same base name (case-sensitive)
- **Check:** Part numbers should be integers (1, 2, 3, not 01, 02, 03)
- **Solution:** Rename files to match supported patterns

Problem: Single file treated as standalone instead of compound

- **Check:** Ensure at least 2 files share the same base name
- **Note:** A group with only 1 part is automatically treated as standalone

Problem: Implicit Part 1 not detected

- **Check:** Ensure `basename.ext` file exists alongside `basename_2.ext, basename_3.ext`
- **Check:** Base name must match exactly (including case)
- **Example:** `Report.pdf + Report_2.pdf` works, but `report.pdf + Report_2.pdf` does not

Problem: Files grouped incorrectly

- **Example:** `Photo_1.jpg` and `Photo_Album_1.jpg` should not group together
- **Check:** Verify exact base name match in filename analysis
- **Workaround:** Rename files to have more distinct base names

Manual Processing Issues (UpdateCSV)

Common Issues:

Problem: "No compound parent/child relationships found"

- **Check:** Verify `compoundrelationship` column exists and has values
- **Check:** Ensure values start with "parent" or "child" (not "Parent" or "PARENT")

Problem: "Too few children" error

- **Check:** Confirm at least 2 child rows immediately follow parent
- **Check:** Verify child rows have `compoundrelationship` starting with "child"

Problem: Empty TOC in parent record

- **Check:** Ensure children have `dc:title` values
- **Check:** Verify `dcterms:tableOfContents` column exists

Problem: Children not linked to parent

- **Check:** Confirm `group_id` column exists in CSV
- **Check:** Verify children immediately follow parent (no gaps)

Related Documentation

- **Alma Digital Documentation:** See Alma Digital help for compound object structure
- **CSV Column Verification:** See [utils.py](#) for column validation logic
- **General UpdateCSV:** See [USER-GUIDE.md](#) for overall UpdateCSV functionality

Contact

For questions about Alma compound object handling in the Manage Digital Ingest application, refer to this document and the related application documentation.